Name: ______________ Cohort/Batch: ______________ Date: ______________ Score: ______________

TERRAFORM PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A DevOps

TOTAL: 10 QUESTIONS

Q1 What problem does Terraform solve in DevOps or infrastructure? *Threat Model*

Q2 What is Terraform's core architecture? *Architecture*

Q3 How does Terraform define infrastructure or configuration as code? *Configuration*

Q4 How does Terraform ensure idempotency? *Hardening*

Q5 How do you handle secrets in Terraform? *SSO / IdP*

Q6 How does Terraform integrate with CI/CD pipelines? *Observability*

Q7 How do you debug a failed Terraform deployment? *Lifecycle*

Q8 What are security best practices for Terraform? *Compliance*

Q9 How does Terraform scale for large environments? *Testing*

Q10 What are common mistakes teams make when adopting Terraform? *Tuning*

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Terraform addresses a specific concern provisioning, — Mitigation

Terraform addresses a specific concern provisioning, configuration management, orchestration, or CI/CD by automating manual steps that are error-prone, slow, or not reproducible when done by hand.

A2 Terraform has a control plane that stores — Primitives

Terraform has a control plane that stores desired state and a data plane (agents or push-based SSH) that enforces it. Understanding this distinction clarifies where configuration is evaluated and where changes are applied.

A3 Terraform uses declarative files (YAML, HCL, or — Hardening

Terraform uses declarative files (YAML, HCL, or a DSL) to express desired state. A plan/diff phase shows what will change before applying. Version-controlling these files enables peer review and rollback.

A4 Terraform operations are designed to produce the — Gotchas

Terraform operations are designed to produce the same result when run multiple times. Applying the same config twice converges to the desired state rather than creating duplicate resources. This makes automation safe to re-run.

A5 Secrets should never be stored in plain — Federation

Secrets should never be stored in plain text in Terraform config files. Use a secrets backend (Vault, AWS Secrets Manager) and inject values at runtime via environment variables or encrypted vaults supported by the tool.

A6 CI/CD pipelines call Terraform in automated steps: — Observability

CI/CD pipelines call Terraform in automated steps: plan on a pull request to preview changes and apply on merge to main. Approval gates can require a human to review the plan before changes go to production.

A7 Check Terraform's logs for the specific error — Automation

Check Terraform's logs for the specific error message and the resource that failed. For infrastructure tools, re-run with increased verbosity (-v, --debug). Review the state file or event history to understand what changed before the failure.

A8 Rotate credentials used by Terraform, restrict network — Governance

Rotate credentials used by Terraform, restrict network access to its control plane, enable audit logging of all operations, and use least-privilege service accounts. Never run Terraform with admin credentials in production.

A9 Large Terraform deployments use workspaces or namespaces — Resilience

Large Terraform deployments use workspaces or namespaces to isolate environments, remote state backends for team collaboration, and modular code to avoid a single monolithic config that is slow to plan/apply.

A10 Common pitfalls include storing secrets in plain — Optimization

Common pitfalls include storing secrets in plain text config, skipping the plan step before apply, not reviewing state drift, and not having a rollback strategy when a deployment fails partway through.